INTERVIEW PREPARATION

SQL Data Analyst Interview Guide

392 Interview Questions with Concise, Interview-Ready Answers

Fundamentals → Joins → Window Functions → Analytics & Business Cases

MySQL-focused • Beginner to Advanced

2026 Edition

Table of Contents

1. SQL & Database Fundamentals (Q1–21) 3
2. Keys and Constraints (Q22–37) 5
3. Basic SQL Queries (Q38–58) 7
4. NULL Values (Q59–71) 9
5. Aggregate Functions (Q72–85) 10
6. GROUP BY and HAVING (Q86–96) 11
7. SQL Joins (Q97–119) 13
8. Subqueries (Q120–132) 15
9. CTEs — Common Table Expressions (Q133–143) 17
10. CASE Statements (Q144–152) 18
11. Window Functions (Q153–175) 19
12. Date and Time Functions (Q176–196) 22
13. String Functions (Q197–215) 25
14. Data Cleaning Using SQL (Q216–235) 26
15. UNION and Set Operations (Q236–245) 29
16. Views (Q246–253) 30
17. Indexes (Q254–264) 31
18. Query Performance & Optimization (Q265–278) 32
19. Transactions (Q279–288) 33
20. Frequently Asked SQL Coding Questions (Q289–300) 34
21. Intermediate SQL Problems (Q301–315) 36
22. Advanced Analytical SQL Questions (Q316–335) 38
23. Business / Data Analyst SQL Questions (Q336–355) 42
24. SQL + Real-World Scenarios (Q356–371) 44
25. MySQL-Specific Questions (Q372–392) 47
26. Skills Checklist & The 25 Questions to Prepare First 49

> *Answers are concise and interview-ready. Coding questions include example SQL (MySQL syntax). Starred questions are the most frequently asked.*

1. SQL & Database Fundamentals

★ 1. What is SQL?

SQL (Structured Query Language) is the standard language used to store, retrieve, manipulate, and manage data in relational databases. It covers querying (SELECT), data changes (INSERT/UPDATE/DELETE), structure changes (CREATE/ALTER), and access control.

★ 2. What is a database?

A database is an organized collection of structured data stored electronically, designed for efficient storage, retrieval, and management — e.g., tables of customers, orders, and products.

★ 3. What is DBMS?

A Database Management System is software that lets users create, read, update, and manage databases. It handles storage, security, backups, and concurrency. Examples: MySQL, Oracle, PostgreSQL.

★ 4. What is RDBMS?

A Relational DBMS stores data in tables (relations) of rows and columns, where tables can be related to each other through keys. It supports SQL and enforces integrity constraints. Examples: MySQL, PostgreSQL, SQL Server.

★ 5. Difference between DBMS and RDBMS?

A DBMS stores data as files or hierarchies with no enforced relationships between data; an RDBMS stores data in related tables with keys, constraints, and normalization, and supports SQL and ACID transactions.

★ 6. What is MySQL?

MySQL is a popular open-source relational database management system that uses SQL. It's widely used in web applications and analytics, known for being fast, reliable, and free.

★ 7. Difference between SQL and MySQL?

SQL is a language; MySQL is a database software product that implements SQL. SQL is the standard for talking to relational databases, while MySQL is one specific RDBMS (like PostgreSQL or SQL Server are others).

8. What is a relational database?

A database organized into tables of rows and columns, where tables relate to each other via primary and foreign keys — e.g., an orders table referencing a customers table by customer_id.

9. What is a table?

A table is a structured collection of related data organized into rows (records) and columns (fields), e.g., an `employees` table with columns id, name, salary.

10. What is a record/row?

A row (record or tuple) is a single entry in a table representing one entity — e.g., one employee with all their attribute values.

11. What is a column?

A column (field/attribute) defines one type of data stored for every row in a table, with a fixed data type — e.g., the `salary` column of type DECIMAL.

12. What is a schema?

A schema is the logical structure/blueprint of a database — its tables, columns, data types, keys, and relationships. In MySQL, "schema" is synonymous with a database.

13. What is a database server?

A database server is the software/machine that hosts the DBMS, listens for client connections, executes queries, and manages data files — e.g., the MySQL server process (mysqld).

14. What are the different types of SQL commands?

Five categories: DDL (structure), DML (data changes), DQL (queries), DCL (permissions), and TCL (transactions).

★ 15. What are DDL, DML, DQL, DCL, and TCL?

DDL (Data Definition): CREATE, ALTER, DROP, TRUNCATE — define structure. **DML** (Data Manipulation): INSERT, UPDATE, DELETE — change data. **DQL** (Data Query): SELECT — retrieve data. **DCL** (Data Control): GRANT, REVOKE — permissions. **TCL** (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT.

16. Difference between structured and unstructured data?

Structured data fits in tables with a defined schema (rows/columns, e.g., sales records). Unstructured data has no predefined model — text, images, videos, logs. Semi-structured (JSON, XML) sits in between.

17. What is normalization?

Normalization is organizing a database into multiple related tables to reduce data redundancy and avoid update anomalies, by applying normal forms (1NF, 2NF, 3NF...).

18. What is denormalization?

Deliberately combining tables or adding redundant data to reduce joins and speed up read queries — common in analytics/reporting where read performance matters more than storage efficiency.

19. What are the different normal forms?

1NF: atomic values, no repeating groups. **2NF**: 1NF + no partial dependency on part of a composite key. **3NF**: 2NF + no transitive dependencies (non-key columns depend only on the key). BCNF is a stricter 3NF.

20. What is data integrity?

The accuracy, consistency, and reliability of data, enforced through constraints: entity integrity (primary keys), referential integrity (foreign keys), domain integrity (data types, CHECK), and user-defined rules.

21. What is referential integrity?

A rule enforced by foreign keys: a foreign key value must either match an existing primary key in the referenced table or be NULL — preventing orphaned records (e.g., an order for a non-existent customer).

2. Keys and Constraints

★ 22. What is a primary key?

A column (or set of columns) that uniquely identifies each row in a table. It must be unique and NOT NULL. Only one primary key per table — e.g., `employee_id`.

★ 23. What is a foreign key?

A column that references the primary key of another (or the same) table, creating a link between them and enforcing referential integrity — e.g., `orders.customer_id` referencing `customers.id`.

★ 24. Difference between PRIMARY KEY and UNIQUE KEY?

Both enforce uniqueness, but a primary key cannot contain NULL and a table can have only one; UNIQUE keys can contain one NULL (MySQL allows multiple) and a table can have many UNIQUE constraints.

25. What is a composite key?

A primary key made of two or more columns combined to uniquely identify a row — e.g., (order_id, product_id) in an order_items table.

26. What is a candidate key?

Any column or set of columns that could uniquely identify rows — a candidate for primary key. A table can have several candidate keys; one is chosen as the primary key.

27. What is a super key?

Any set of one or more columns that uniquely identifies a row, including combinations with extra (unnecessary) columns. Candidate keys are minimal super keys.

28. What is a surrogate key?

An artificial key with no business meaning, usually an auto-increment integer, created solely to uniquely identify rows — e.g., an AUTO_INCREMENT id.

29. What is a natural key?

A key formed from real-world business attributes — e.g., email address, SSN, or ISBN — as opposed to an artificial surrogate key.

★ 30. Can a primary key contain NULL values?

No. A primary key must be UNIQUE and NOT NULL by definition — every row must be uniquely identifiable.

31. Can a table have multiple primary keys?

No — only one primary key per table. However, that single primary key can be composite (span multiple columns).

32. Can a table have multiple UNIQUE keys?

Yes. A table can have many UNIQUE constraints on different columns (e.g., unique email and unique phone), but only one primary key.

33. What is a NOT NULL constraint?

It ensures a column cannot store NULL — every row must have a value for that column.

34. What is a DEFAULT constraint?

It assigns a default value to a column when no value is provided on INSERT — e.g., `created_at DATETIME DEFAULT CURRENT_TIMESTAMP`.

35. What is a CHECK constraint?

It limits the values allowed in a column based on a condition — e.g., `CHECK (salary > 0)` rejects non-positive salaries.

36. What is a constraint?

A rule applied to columns/tables to enforce data accuracy and integrity: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK.

37. Why are foreign keys important?

They maintain referential integrity — preventing invalid references (orphaned rows), enabling cascading updates/deletes, and documenting relationships between tables.

3. Basic SQL Queries

★ 38. How do you retrieve all records from a table?

```
SELECT * FROM employees;
```

★ 39. How do you select specific columns?

```
SELECT first_name, salary FROM employees;
```

★ 40. Difference between SELECT * and selecting specific columns?

`SELECT *` returns every column — convenient for exploration but slower, reads more data than needed, and can break if the schema changes. Naming columns is faster, clearer, and safer in production queries.

★ 41. How does the WHERE clause work?

WHERE filters rows *before* any grouping or aggregation, keeping only rows where the condition is true: `SELECT * FROM employees WHERE salary > 50000;`

42. Difference between WHERE and HAVING?

WHERE filters individual rows before grouping and cannot use aggregate functions; HAVING filters groups after GROUP BY and can use aggregates (e.g., `HAVING COUNT(*) > 10`).

43. What is the DISTINCT keyword?

It removes duplicate rows from the result set: `SELECT DISTINCT department FROM employees;`

★ 44. How do you find duplicate values?

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

45. How do you sort data?

With ORDER BY: `SELECT * FROM employees ORDER BY salary DESC;` (ASC is the default).

★ 46. What is ORDER BY?

A clause that sorts the result set by one or more columns in ascending (ASC, default) or descending (DESC) order. You can sort by multiple columns: `ORDER BY dept, salary DESC`.

47. Difference between ascending and descending order?

Ascending (ASC) sorts smallest to largest / A–Z (default); descending (DESC) sorts largest to smallest / Z–A.

48. What does LIMIT do?

It restricts the number of rows returned: `SELECT * FROM orders LIMIT 10;` returns only 10 rows. `LIMIT 10 OFFSET 20` skips 20 rows first (used for pagination).

49. How do you retrieve the top 5 records?

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

50. How do you retrieve the second-highest value?

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

51. How do you retrieve the top 10 highest salaries?

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 10;
```

52. How do you filter records using multiple conditions?

Combine conditions with AND/OR and parentheses: `WHERE dept = 'Sales' AND salary > 50000 OR dept = 'IT'` — use parentheses to control precedence.

53. What are AND, OR, and NOT?

Logical operators in WHERE: AND requires all conditions true; OR requires at least one; NOT negates a condition. AND has higher precedence than OR — use parentheses for clarity.

54. What is the IN operator?

It checks whether a value matches any value in a list — shorthand for multiple ORs: `WHERE dept IN ('Sales', 'IT', 'HR')`.

55. What is the BETWEEN operator?

It checks whether a value lies within an inclusive range: `WHERE salary BETWEEN 40000 AND 80000` (both endpoints included).

56. What is the LIKE operator?

It matches strings against a pattern using wildcards: `WHERE name LIKE 'A%'` finds names starting with A.

57. What are wildcards in SQL?

Special characters used with LIKE: `%` matches any sequence of characters (including none), and `_` matches exactly one character.

58. Difference between % and _ in LIKE?

`%` = zero or more characters (`'Jo%'` matches John, Jordan, Jo); `_` = exactly one character (`'J_hn'` matches John).

4. NULL Values

★ 59. What is NULL in SQL?

NULL represents a missing or unknown value. It is not zero, not an empty string, and not equal to anything — not even another NULL.

★ 60. Difference between NULL and 0?

0 is an actual numeric value you can compare and calculate with; NULL means "no value / unknown". NULL + anything = NULL, while 0 behaves normally in arithmetic.

★ 61. Difference between NULL and an empty string?

An empty string ('') is a real string of length 0; NULL means the value is unknown/missing. `col = ''` matches empty strings but never NULLs — NULLs need `IS NULL`.

62. How do you find NULL values?

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

63. How do you find non-NULL values?

```
SELECT * FROM employees WHERE manager_id IS NOT NULL;
```

★ 64. What is IS NULL?

The correct operator to test for NULL, because `= NULL` never returns true (NULL comparisons evaluate to UNKNOWN).

★ 65. What is IS NOT NULL?

The operator that returns rows where the column has an actual (non-NULL) value.

66. What is COALESCE()?

It returns the first non-NULL value from its arguments: `COALESCE(phone, mobile, 'N/A')`. Standard SQL, works with any number of arguments.

67. What is IFNULL() in MySQL?

A MySQL-specific function with exactly two arguments: `IFNULL(col, 'default')` returns the column value, or the default if it's NULL.

68. Difference between COALESCE() and IFNULL()?

IFNULL takes exactly two arguments and is MySQL-specific; COALESCE takes any number of arguments, is ANSI-standard, and works across all databases.

69. How do NULL values behave in calculations?

Any arithmetic involving NULL yields NULL (10 + NULL = NULL). NULL comparisons evaluate to UNKNOWN, so rows with NULL are filtered out of WHERE conditions.

70. How does NULL affect aggregate functions?

SUM, AVG, MIN, MAX and COUNT(column) ignore NULLs; COUNT(*) counts all rows including NULLs. AVG divides by the count of non-NULL values only.

71. How would you replace NULL values?

```
SELECT name, COALESCE(salary, 0) AS salary FROM employees;
-- or permanently:
UPDATE employees SET salary = 0 WHERE salary IS NULL;
```

5. Aggregate Functions

★ 72. What are aggregate functions?

Functions that perform a calculation across a set of rows and return a single summary value: COUNT, SUM, AVG, MIN, MAX. Usually combined with GROUP BY.

★ 73. What is COUNT()?

Counts rows. `COUNT(*)` counts all rows; `COUNT(column)` counts non-NULL values in that column.

★ 74. Difference between COUNT(*) and COUNT(column)?

`COUNT(*)` counts every row including those with NULLs; `COUNT(column)` counts only rows where that column is not NULL. This is a classic interview trap.

75. What is COUNT(DISTINCT column)?

Counts the number of unique non-NULL values: `SELECT COUNT(DISTINCT customer_id) FROM orders;`

★ 76. What is SUM()?

Returns the total of numeric values in a column, ignoring NULLs: `SELECT SUM(amount) FROM sales;`

★ 77. What is AVG()?

Returns the average of non-NULL values: `SELECT AVG(salary) FROM employees;`

★ 78. What is MIN()?

Returns the smallest value in a column (works on numbers, dates, and strings).

★ 79. What is MAX()?

Returns the largest value in a column (works on numbers, dates, and strings).

80. How do aggregate functions handle NULL values?

All aggregates ignore NULLs except COUNT(*). AVG computes sum of non-NULL values divided by count of non-NULL values.

81. How do you calculate the average salary?

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

82. How do you find the maximum salary?

```
SELECT MAX(salary) FROM employees;
```

83. How do you find the minimum salary?

```
SELECT MIN(salary) FROM employees;
```

84. How do you count unique customers?

```
SELECT COUNT(DISTINCT customer_id) FROM orders;
```

85. How do you calculate total sales?

```
SELECT SUM(amount) AS total_sales FROM orders;
```

6. GROUP BY and HAVING

★ 86. What is GROUP BY?

It groups rows sharing the same values into summary rows, so aggregate functions can be applied per group — e.g., total sales per region.

★ 87. Why do we use GROUP BY?

To compute aggregates per category rather than over the whole table — employees per department, revenue per month, orders per customer.

★ 88. Difference between WHERE and HAVING?

WHERE filters rows before grouping and can't reference aggregates; HAVING filters groups after aggregation. Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

89. Can we use WHERE with aggregate functions?

No — WHERE runs before aggregation, so `WHERE COUNT(*) > 5` is invalid. Use HAVING for aggregate conditions.

90. Can we use HAVING without GROUP BY?

Yes — the whole result is then treated as one group, e.g. `SELECT AVG(salary) FROM employees HAVING AVG(salary) > 50000;` (rare in practice).

★ 91. How do you find the number of employees in each department?

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

92. How do you find departments having more than 10 employees?

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 10;
```

93. How do you find the average salary by department?

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

94. How do you find the highest salary in each department?

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

95. How do you find customers who placed more than 5 orders?

```
SELECT customer_id, COUNT(*) AS orders_cnt
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

96. How do you group data by multiple columns?

```
SELECT department, job_title, COUNT(*)
FROM employees
GROUP BY department, job_title;
```

7. SQL Joins

★ 97. What is a JOIN?

A JOIN combines rows from two or more tables based on a related column (usually a primary/foreign key relationship).

★ 98. Why are joins used?

Because normalized data is spread across multiple tables; joins let you reconstruct related information — e.g., orders together with customer names.

★ 99. What is an INNER JOIN?

Returns only rows with matching values in both tables. Non-matching rows from either side are excluded.

★ 100. What is a LEFT JOIN?

Returns all rows from the left table plus matching rows from the right; where no match exists, right-side columns are NULL.

★ 101. What is a RIGHT JOIN?

Returns all rows from the right table plus matches from the left; unmatched right rows show NULL on the left side. It's a LEFT JOIN with tables swapped.

102. What is a FULL OUTER JOIN?

Returns all rows from both tables, matching where possible and filling NULLs where not. Not supported directly in MySQL — emulate with LEFT JOIN UNION RIGHT JOIN.

★ 103. What is a CROSS JOIN?

Produces the Cartesian product — every row of the first table paired with every row of the second (m × n rows). Used for generating combinations/date grids.

104. What is a SELF JOIN?

Joining a table to itself using aliases — e.g., matching employees to their managers stored in the same employees table.

★ 105. Difference between INNER JOIN and LEFT JOIN?

INNER JOIN keeps only matching rows; LEFT JOIN keeps all left-table rows even with no match (NULLs on the right).

★ 106. Difference between LEFT JOIN and RIGHT JOIN?

They mirror each other: LEFT keeps all rows from the left table; RIGHT keeps all rows from the right table. Any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping table order.

107. When would you use a LEFT JOIN?

When you must keep every row of the primary table regardless of matches — e.g., all customers and their orders (if any), or to find records that have no match.

108. How do you join three tables?

```
SELECT c.name, o.order_id, p.product_name
FROM customers c
JOIN orders o   ON o.customer_id = c.id
JOIN products p ON p.id = o.product_id;
```

109. Can you join a table with itself?

Yes — a self join using two aliases:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

110. What happens when there is no matching record in a JOIN?

INNER JOIN: the row disappears. LEFT/RIGHT/FULL JOIN: the row is kept and the other side's columns become NULL.

★ 111. How do you find records that exist in one table but not another?

```
SELECT a.*
FROM table_a a
LEFT JOIN table_b b ON a.id = b.id
WHERE b.id IS NULL;
```

112. How do you find customers who have never placed an order?

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.customer_id IS NULL;
```

113. How do you find employees who don't belong to any department?

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;
-- or simply: WHERE e.dept_id IS NULL
```

114. How do you identify unmatched records?

Use a LEFT JOIN and filter WHERE the right table's key IS NULL; for both directions, use UNION of LEFT and RIGHT anti-joins (or FULL OUTER JOIN where supported).

115. What is a many-to-many relationship?

Records on both sides relate to many records on the other side (students ↔ courses). It requires a junction/bridge table with two foreign keys.

116. What is a one-to-many relationship?

One record in the parent table relates to many in the child (one customer → many orders). Implemented with a foreign key on the "many" side.

117. What is a one-to-one relationship?

Each record in one table matches at most one record in the other (person ↔ passport). Implemented with a UNIQUE foreign key.

118. How can joins create duplicate rows?

When the join key is not unique on one side (one-to-many), each matching row multiplies — e.g., joining customers to orders repeats the customer once per order, inflating sums.

119. How do you avoid duplicate rows after a JOIN?

Use DISTINCT, aggregate the many-side first in a subquery/CTE before joining, join on unique keys, or deduplicate with ROW_NUMBER() and keep one row per key.

8. Subqueries

★ 120. What is a subquery?

A query nested inside another SQL statement, used to supply a value, list, or derived table to the outer query.

★ 121. Difference between a subquery and a JOIN?

A JOIN combines columns from multiple tables into one row set; a subquery computes an intermediate result (value/list/table) used by the outer query. Joins are usually faster for combining data; subqueries can be clearer for filtering logic.

122. What is a correlated subquery?

A subquery that references columns from the outer query, so it is re-evaluated once per outer row — e.g., comparing each employee's salary to their own department's average. Usually slower.

123. What is a non-correlated subquery?

A subquery independent of the outer query — it executes once and its result is reused: `WHERE salary > (SELECT AVG(salary) FROM employees)`.

124. Where can you use a subquery?

In SELECT (scalar value), FROM (derived table), WHERE/HAVING (filter), and with IN, EXISTS, ANY, ALL, and comparison operators.

125. Can a subquery be used in SELECT?

Yes, as a scalar subquery returning one value: `SELECT name, (SELECT AVG(salary) FROM employees) AS company_avg FROM employees;`

126. Can a subquery be used in FROM?

Yes — it becomes a derived table (inline view) and must have an alias: `SELECT * FROM (SELECT dept, AVG(salary) a FROM employees GROUP BY dept) t WHERE a > 50000;`

127. Can a subquery be used in WHERE?

Yes, the most common use: `WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount > 1000)`.

★ 128. Difference between IN and EXISTS?

IN compares a value against a materialized list returned by the subquery; EXISTS checks whether the subquery returns any row and stops at the first match. EXISTS is usually faster for large subquery results and handles NULLs more safely (NOT IN with a NULL in the list returns no rows).

★ 129. When would you use EXISTS?

When you only care whether at least one matching row exists — e.g., customers who have placed at least one order — especially with correlated subqueries on large tables, and for NOT EXISTS anti-joins (NULL-safe).

130. How do you find employees earning more than the average salary?

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

131. How do you find the second-highest salary using a subquery?

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

132. How do you find customers whose spending is above the average?

```
SELECT customer_id, SUM(amount) AS total
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > (
    SELECT AVG(total) FROM (
        SELECT SUM(amount) AS total FROM orders GROUP BY customer_id
    ) t
);
```

9. CTEs — Common Table Expressions

★ 133. What is a CTE?

A named temporary result set defined with WITH, available for the duration of one query: `WITH dept_avg AS (SELECT dept, AVG(salary) a FROM employees GROUP BY dept) SELECT * FROM dept_avg;`

★ 134. Why do we use CTEs?

To break complex logic into readable, named steps; avoid repeating subqueries; enable recursion; and make queries easier to debug and maintain.

135. What is the syntax of a CTE?

```
WITH cte_name AS (
    SELECT ...
)
SELECT * FROM cte_name;
```

136. Difference between a CTE and a subquery?

A subquery is an anonymous nested query; a CTE is named, defined once at the top, can be referenced multiple times, can reference other CTEs, and supports recursion. CTEs are generally more readable.

137. What are the advantages of CTEs?

Readability, reusability within the query, step-by-step layered logic, easier debugging, recursive capability, and a cleaner alternative to deeply nested subqueries.

138. Can we use multiple CTEs?

Yes, separate them with commas after one WITH keyword: `WITH a AS (...), b AS (...) SELECT ...`.

139. Can a CTE reference another CTE?

Yes — a CTE can use any CTE defined before it in the same WITH clause, enabling layered transformations.

140. What is a recursive CTE?

A CTE that references itself, with an anchor member and a recursive member, iterating until no new rows are produced: `WITH RECURSIVE seq AS (SELECT 1 n UNION ALL SELECT n+1 FROM seq WHERE n < 10) SELECT * FROM seq;`

141. When would you use a recursive CTE?

For hierarchical or iterative data: org charts (manager → employee trees), bill of materials, date series generation, and graph traversal.

142. How can CTEs make SQL queries easier to understand?

Each step gets a descriptive name (e.g., `monthly_sales`, `top_customers`), so the query reads top-down like a pipeline instead of inside-out nested subqueries.

143. How would you use a CTE to calculate the top 3 customers?

```
WITH customer_totals AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT * FROM customer_totals
ORDER BY total_spent DESC
LIMIT 3;
```

10. CASE Statements

★ 144. What is a CASE statement?

SQL's conditional logic (like IF/ELSE) inside a query. It evaluates conditions and returns a value: `CASE WHEN salary > 100000 THEN 'High' ELSE 'Low' END`.

★ 145. Why is CASE used in SQL?

For categorization/labeling, conditional aggregation (count or sum only rows meeting a condition), pivoting rows into columns, and custom sorting logic.

146. Difference between simple CASE and searched CASE?

Simple CASE compares one expression to values: `CASE col WHEN 'A' THEN ... END`. Searched CASE allows independent conditions: `CASE WHEN x > 5 THEN ... WHEN y < 2 THEN ... END` — more flexible.

147. How do you categorize customers based on spending?

```
SELECT customer_id, total_spent,
  CASE
    WHEN total_spent >= 10000 THEN 'VIP'
    WHEN total_spent >= 3000  THEN 'Regular'
    ELSE 'Occasional'
  END AS segment
FROM customer_totals;
```

148. How do you create an age group using CASE?

```
CASE
  WHEN age < 18 THEN 'Under 18'
  WHEN age BETWEEN 18 AND 34 THEN '18-34'
  WHEN age BETWEEN 35 AND 54 THEN '35-54'
  ELSE '55+'
END AS age_group
```

149. How do you classify employees as High/Medium/Low salary?

```
CASE
  WHEN salary > 90000 THEN 'High'
  WHEN salary > 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_band
```

150. How do you count records conditionally using CASE?

```
SELECT
  SUM(CASE WHEN gender = 'Male'   THEN 1 ELSE 0 END) AS male_count,
  SUM(CASE WHEN gender = 'Female' THEN 1 ELSE 0 END) AS female_count
FROM employees;
```

151. How do you calculate conditional percentages?

```
SELECT
  100.0 * SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) / COUNT(*)
    AS completion_pct
FROM orders;
```

152. How can CASE be used with aggregate functions?

CASE sits inside the aggregate to filter which rows contribute — e.g., `SUM(CASE WHEN region='West' THEN amount END)` gives West sales while the query still returns one row overall. This is the standard pivoting technique.

11. Window Functions

★ 153. What is a window function?

A function that performs a calculation across a set of rows related to the current row (a "window") defined by OVER(), while keeping every row in the result — e.g., `ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)`.

★ 154. How is a window function different from GROUP BY?

GROUP BY collapses rows into one row per group; a window function computes a value per group but *keeps all original rows*, attaching the result to each row. You can also mix both.

★ 155. What is ROW_NUMBER()?

Assigns a unique sequential number (1,2,3...) to each row within a partition according to ORDER BY — ties get arbitrary distinct numbers.

★ 156. What is RANK()?

Assigns the same rank to ties, then skips the following ranks: 1, 2, 2, 4.

★ 157. What is DENSE_RANK()?

Like RANK but without gaps: 1, 2, 2, 3.

★ 158. Difference between RANK() and DENSE_RANK()?

Both give ties the same rank; RANK skips numbers after ties (1,2,2,4), DENSE_RANK doesn't (1,2,2,3). Use DENSE_RANK for "top N distinct values per group".

★ 159. Difference between ROW_NUMBER() and RANK()?

ROW_NUMBER always produces unique consecutive numbers even for ties; RANK gives ties the same number and skips ahead after them. ROW_NUMBER is used to pick exactly N rows per group; RANK keeps ties.

160. What is PARTITION BY?

The clause inside OVER() that divides rows into groups (like GROUP BY without collapsing). The window function restarts its calculation for each partition.

161. What is ORDER BY inside a window function?

It defines the row ordering within each partition used by ranking, LAG/LEAD, and cumulative calculations: `OVER (PARTITION BY dept ORDER BY salary DESC)`.

162. What is LEAD()?

Returns a column value from the *next* row in the partition: `LEAD(amount) OVER (ORDER BY order_date)` — used to compare with the following period.

163. What is LAG()?

Returns a column value from the *previous* row in the partition — used for month-over-month comparisons and gaps between events.

164. What is FIRST_VALUE()?

Returns the value from the first row of the window frame — e.g., the highest salary in each department attached to every employee row.

165. What is LAST_VALUE()?

Returns the value from the last row of the window frame. Note: with the default frame you often need `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the true last value.

166. What is a running total?

A cumulative sum where each row shows the total of all rows up to and including itself — e.g., sales-to-date per day.

167. How do you calculate a running total?

```
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM sales;
```

168. How do you calculate a moving average?

```
SELECT order_date, amount,
  AVG(amount) OVER (
    ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7d
FROM sales;
```

169. How do you find the top 3 employees in each department?

```
SELECT * FROM (
  SELECT e.*,
    ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
  FROM employees e
) t
WHERE rn <= 3;
```

★ 170. How do you find the second-highest salary in each department?

```
SELECT dept, salary FROM (
  SELECT dept, salary,
    DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk = 2;
```

171. How do you compare current sales with previous month's sales?

```
SELECT month, sales,
  LAG(sales) OVER (ORDER BY month) AS prev_month_sales
FROM monthly_sales;
```

172. How do you calculate month-over-month growth?

```
SELECT month, sales,
  100.0 * (sales - LAG(sales) OVER (ORDER BY month))
    / LAG(sales) OVER (ORDER BY month) AS mom_growth_pct
FROM monthly_sales;
```

173. How do you calculate cumulative sales?

```
SELECT month, sales,
  SUM(sales) OVER (ORDER BY month) AS cumulative_sales
FROM monthly_sales;
```

174. How do you identify the first order of each customer?

```
SELECT * FROM (
  SELECT o.*,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
  FROM orders o
) t WHERE rn = 1;
```

175. How do you identify the latest order of each customer?

Same pattern with `ORDER BY order_date DESC` and keep rn = 1; or filter `WHERE order_date = (SELECT MAX(order_date) FROM orders o2 WHERE o2.customer_id = o.customer_id)`.

12. Date and Time Functions

★ 176. How do you work with dates in SQL?

Store them as DATE/DATETIME/TIMESTAMP, extract parts with YEAR()/MONTH()/DAY(), compute differences with DATEDIFF()/TIMESTAMPDIFF(), shift with DATE_ADD()/DATE_SUB(), and format with DATE_FORMAT().

★ 177. What is DATE()?

Extracts the date portion from a datetime value: `DATE('2026-10-08 14:30:00')` → '2026-10-08'.

178. What is YEAR()?

Returns the year part of a date: `YEAR('2026-10-08')` → 2026.

179. What is MONTH()?

Returns the month number (1–12) of a date.

180. What is DAY()?

Returns the day of the month (1–31) of a date.

181. What is MONTHNAME()?

Returns the month name: `MONTHNAME('2026-10-08')` → 'October'.

182. What is DAYNAME()?

Returns the weekday name: `DAYNAME('2026-10-08')` → 'Thursday'.

183. What is DATEDIFF()?

Returns the number of days between two dates: `DATEDIFF('2026-10-08','2026-10-01')` → 7.

184. What is DATE_ADD()?

Adds an interval to a date: `DATE_ADD(order_date, INTERVAL 30 DAY)`.

185. What is DATE_SUB()?

Subtracts an interval: `DATE_SUB(CURDATE(), INTERVAL 7 DAY)`.

186. What is TIMESTAMPDIFF()?

Returns the difference between two datetimes in a chosen unit: `TIMESTAMPDIFF(MONTH, start_date, end_date)` — works with YEAR, MONTH, DAY, HOUR, MINUTE, SECOND.

187. How do you extract the year from a date?

```
SELECT YEAR(order_date) FROM orders;
-- or EXTRACT(YEAR FROM order_date)
```

188. How do you extract the month?

```
SELECT MONTH(order_date) FROM orders;
```

189. How do you calculate the difference between two dates?

```
SELECT DATEDIFF(end_date, start_date) AS days_diff FROM projects;
```

190. How do you find records from the last 30 days?

```
SELECT * FROM orders
WHERE order_date >= CURDATE() - INTERVAL 30 DAY;
```

191. How do you find customers who purchased this month?

```
SELECT DISTINCT customer_id FROM orders
WHERE YEAR(order_date) = YEAR(CURDATE())
  AND MONTH(order_date) = MONTH(CURDATE());
```

192. How do you calculate monthly sales?

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS monthly_sales
FROM orders
GROUP BY month
ORDER BY month;
```

193. How do you calculate yearly sales?

```
SELECT YEAR(order_date) AS yr, SUM(amount) AS yearly_sales
FROM orders GROUP BY yr ORDER BY yr;
```

194. How do you find the busiest month?

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, COUNT(*) AS orders_cnt
FROM orders
GROUP BY month
ORDER BY orders_cnt DESC
LIMIT 1;
```

195. How do you calculate customer retention by month?

Build monthly cohorts of first-purchase month, then count how many from each cohort ordered again in each later month, dividing by cohort size:

```
WITH cohort AS (
  SELECT customer_id, MIN(DATE_FORMAT(order_date,'%Y-%m')) AS cohort_month
  FROM orders GROUP BY customer_id
)
SELECT c.cohort_month,
       DATE_FORMAT(o.order_date,'%Y-%m') AS active_month,
       COUNT(DISTINCT o.customer_id) AS active_customers
FROM orders o JOIN cohort c ON o.customer_id = c.customer_id
GROUP BY c.cohort_month, active_month;
```

196. How do you calculate the average number of days between orders?

```
WITH gaps AS (
  SELECT customer_id,
    DATEDIFF(order_date,
      LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
    ) AS days_between
  FROM orders
)
SELECT customer_id, AVG(days_between) AS avg_days_between_orders
FROM gaps WHERE days_between IS NOT NULL
GROUP BY customer_id;
```

13. String Functions

★ 197. What are string functions?

Built-in functions to manipulate text: combining (CONCAT), measuring (LENGTH), changing case (UPPER/LOWER), trimming (TRIM), extracting (SUBSTRING, LEFT, RIGHT), and replacing (REPLACE).

198. What is CONCAT()?

Joins strings together: `CONCAT(first_name, ' ', last_name)`. Note: returns NULL if any argument is NULL.

199. What is CONCAT_WS()?

CONCAT With Separator: `CONCAT_WS(', ', city, state, country)` — it inserts the separator and skips NULL arguments.

200. What is LENGTH()?

Returns the length of a string in bytes (use CHAR_LENGTH for character count with multi-byte text).

201. What is LOWER()?

Converts a string to lowercase: `LOWER('SQL')` → 'sql'.

202. What is UPPER()?

Converts a string to uppercase.

203. What is TRIM()?

Removes leading and trailing spaces: `TRIM(' hello ')` → 'hello'.

204. What is LTRIM()?

Removes leading (left-side) spaces only.

205. What is RTRIM()?

Removes trailing (right-side) spaces only.

206. What is SUBSTRING()?

Extracts part of a string: `SUBSTRING('Database', 1, 4)` → 'Data' (start position, length).

207. What is LEFT()?

Returns the leftmost N characters: `LEFT('Analytics', 4)` → 'Anal'.

208. What is RIGHT()?

Returns the rightmost N characters.

209. What is REPLACE()?

Replaces all occurrences of a substring: `REPLACE(phone, '-', '')` strips dashes.

210. What is CHAR_LENGTH()?

Returns the number of characters (not bytes) in a string — important for multi-byte/Unicode text.

211. How do you remove leading/trailing spaces?

```
UPDATE customers SET name = TRIM(name);
```

212. How do you extract part of a string?

```
SELECT SUBSTRING(email, LOCATE('@', email) + 1) AS domain
FROM users;
```

213. How do you combine first name and last name?

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

214. How do you replace a particular character?

```
SELECT REPLACE(product_code, '_', '-') FROM products;
```

215. How do you identify duplicate names with different capitalization?

```
SELECT LOWER(TRIM(name)) AS norm_name, COUNT(*) AS cnt
FROM customers
GROUP BY norm_name
HAVING cnt > 1;
```

14. Data Cleaning Using SQL

★ 216. How do you identify duplicate records?

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING cnt > 1;
```

★ 217. How do you remove duplicate records?

```
-- keep the lowest id per email
DELETE c FROM customers c
JOIN (
  SELECT email, MIN(id) AS keep_id
  FROM customers GROUP BY email
) k ON c.email = k.email AND c.id > k.keep_id;
```

★ 218. How do you handle NULL values?

First profile them (`WHERE col IS NULL`), then decide: filter out, replace with defaults via COALESCE/IFNULL, impute with an aggregate (e.g., AVG), or fix at the source. Document the choice.

219. How do you find inconsistent values?

```
SELECT DISTINCT LOWER(TRIM(status)) FROM orders;
-- reveals variants like 'Shipped', 'shipped ', 'SHIPPED'
```

220. How do you remove leading/trailing spaces?

```
UPDATE customers SET name = TRIM(name);
```

221. How do you standardize text values?

Apply TRIM + consistent case + mapping, e.g. `UPDATE orders SET status = 'shipped' WHERE LOWER(TRIM(status)) IN ('shipped','shiped');` Ideally enforce with a lookup table or ENUM/CHECK.

222. How do you convert text to lowercase?

```
SELECT LOWER(name) FROM customers;
```

223. How do you convert text to uppercase?

```
SELECT UPPER(name) FROM customers;
```

224. How do you replace incorrect values?

```
UPDATE customers SET country = 'USA'
WHERE country IN ('US', 'U.S.A', 'United States');
```

225. How do you identify invalid dates?

If dates are stored as text, test with STR_TO_DATE: `SELECT * FROM t WHERE date_str IS NOT NULL AND STR_TO_DATE(date_str, '%Y-%m-%d') IS NULL;` Proper DATE columns can't hold invalid dates.

226. How do you find outliers using SQL?

Common approaches: values beyond mean ± 3 standard deviations, or outside the IQR fences (Q1 − 1.5×IQR, Q3 + 1.5×IQR):

```
SELECT * FROM sales
WHERE amount > (SELECT AVG(amount) + 3*STDDEV(amount) FROM sales);
```

227. How do you check the uniqueness of a column?

```
SELECT COUNT(*) AS total, COUNT(DISTINCT id) AS unique_ids
FROM users;  -- equal => unique
```

228. How do you identify missing values?

```
SELECT
  SUM(email IS NULL)     AS missing_email,
  SUM(phone IS NULL)     AS missing_phone,
  SUM(TRIM(name) = '')   AS blank_names
FROM customers;
```

229. How do you split a column into multiple columns?

```
SELECT
  SUBSTRING_INDEX(full_name, ' ', 1)  AS first_name,
  SUBSTRING_INDEX(full_name, ' ', -1) AS last_name
FROM customers;
```

230. How do you combine multiple columns?

```
SELECT CONCAT_WS(', ', city, state, zip) AS address FROM customers;
```

231. How do you convert a string into a date?

```
SELECT STR_TO_DATE('08/10/2026', '%d/%m/%Y');
```

232. How do you convert a string into a number?

```
SELECT CAST('123.45' AS DECIMAL(10,2));
```

233. What is CAST()?

Standard SQL function converting a value to another data type: `CAST(col AS UNSIGNED)`, `CAST(price AS DECIMAL(10,2))`.

234. What is CONVERT()?

MySQL's alternative type-conversion function: `CONVERT(col, CHAR)`; also used for character sets: `CONVERT(col USING utf8mb4)`.

235. How would you clean a messy dataset using SQL?

Typical pipeline: (1) profile — count NULLs, duplicates, distinct values; (2) trim and standardize text; (3) fix invalid types with CAST/STR_TO_DATE; (4) remove or flag duplicates with ROW_NUMBER; (5) handle

NULLs via COALESCE or deletion; (6) validate ranges and referential integrity. Do it stepwise in CTEs or staging tables so each step is auditable.

15. UNION and Set Operations

★ 236. What is UNION?

Combines the result sets of two queries vertically and removes duplicate rows. Both queries must have the same number of compatible columns.

★ 237. What is UNION ALL?

Combines result sets like UNION but keeps all duplicates — faster because it skips the deduplication step.

★ 238. Difference between UNION and UNION ALL?

UNION removes duplicates (extra sorting/hashing work, slower); UNION ALL keeps everything (faster). Use UNION ALL when duplicates are impossible or acceptable.

239. What is INTERSECT?

Returns only rows present in both result sets. (Supported in PostgreSQL/SQL Server; in MySQL 8.0.31+, or emulate with INNER JOIN.)

240. What is EXCEPT?

Returns rows from the first query that are not in the second (called MINUS in Oracle). Emulate in MySQL with LEFT JOIN ... IS NULL or NOT EXISTS.

241. When would you use UNION?

When stacking results from two sources where overlap is possible and you want each row once — e.g., combining current and archived customers.

242. When would you use UNION ALL?

When the sources can't overlap (different time periods/regions) or you deliberately want to keep duplicates — faster than UNION.

243. What conditions must be satisfied when using UNION?

Both SELECTs must have the same number of columns, in the same order, with compatible data types. Column names come from the first SELECT.

244. Does UNION remove duplicates?

Yes.

245. Does UNION ALL remove duplicates?

No — it returns all rows from both queries.

16. Views

★ 246. What is a View?

A saved SQL query that behaves like a virtual table. It stores no data itself — each use re-runs the underlying query: `CREATE VIEW v_sales AS SELECT ...;`

247. Why do we use Views?

To simplify complex queries for end users, enforce security (expose only certain columns/rows), provide a stable interface, and reuse common logic.

248. Difference between a table and a View?

A table physically stores data; a view stores only a query definition and computes results on demand. Views always show current data but add query overhead.

249. What is a materialized view?

A view whose results are physically stored (cached) and refreshed periodically — fast reads but potentially stale data. Supported in PostgreSQL/Oracle; in MySQL you emulate it with a summary table plus scheduled refresh.

250. Can we update a View?

Only simple views based on one table without aggregates, DISTINCT, GROUP BY, or joins are updatable; the changes propagate to the base table.

251. What are the advantages of Views?

Simplicity, security (row/column-level restriction), reusability, logical data independence, and consistent business logic.

252. What are the disadvantages of Views?

No stored data (recomputed each time, can be slow), limited updatability, dependency on base table structure, and hidden complexity when views nest views.

253. When would you create a View for a Data Analyst project?

To give stakeholders a clean, pre-joined, pre-filtered dataset (e.g., a `v_monthly_revenue` view powering a dashboard) so everyone queries consistent logic without rewriting complex SQL.

17. Indexes

★ 254. What is an index?

A data structure (usually a B-tree) that lets the database find rows quickly without scanning the whole table — like a book's index.

★ 255. Why are indexes used?

To speed up WHERE filters, JOINs, ORDER BY, and uniqueness checks on large tables, trading storage and write speed for read speed.

256. How does an index improve query performance?

The engine navigates the B-tree (O(log n)) directly to matching rows instead of reading every row (full table scan), and can also satisfy sorts and lookups from the index alone.

257. What are the disadvantages of indexes?

They consume disk space and slow down INSERT/UPDATE/DELETE because every index must be maintained; too many indexes also confuse the optimizer.

258. What is a clustered index?

An index where the table's rows are physically stored in index order (in MySQL InnoDB, the primary key is the clustered index). Only one per table.

259. What is a non-clustered index?

A separate structure holding indexed column values plus pointers to the actual rows. A table can have many of them.

260. Can a table have multiple indexes?

Yes — one clustered (primary key) and many non-clustered (secondary) indexes.

261. Can indexes slow down INSERT/UPDATE operations?

Yes — every write must update all affected indexes, so heavy indexing hurts write-heavy workloads.

262. Which columns should be indexed?

Columns frequently used in WHERE, JOIN conditions, and ORDER BY/GROUP BY, with high selectivity (many distinct values), on large tables. Avoid indexing low-cardinality columns (e.g., boolean flags).

263. What is a composite index?

An index on multiple columns, e.g. `INDEX(dept, salary)`. It follows the leftmost-prefix rule: it helps queries filtering on dept, or dept+salary, but not salary alone.

264. What is a unique index?

An index that also enforces uniqueness of the indexed column(s), combining fast lookup with a data-integrity constraint.

18. Query Performance & Optimization

★ **265. How do you optimize a SQL query?**

(1) Read the EXPLAIN plan; (2) add appropriate indexes on filter/join columns; (3) select only needed columns; (4) filter early; (5) avoid functions on indexed columns in WHERE; (6) rewrite correlated subqueries as joins/CTEs; (7) aggregate before joining; (8) use UNION ALL when possible.

★ **266. Why should you avoid SELECT *?**

It reads and transfers unneeded columns (more I/O and network), prevents covering-index use, breaks downstream code when schema changes, and hides what the query actually needs.

267. What is an execution plan?

The database's step-by-step strategy for running your query — join order, index usage, scan types — revealed by EXPLAIN. It's the primary tool for diagnosing slow queries.

268. What is EXPLAIN in MySQL?

A command (`EXPLAIN SELECT ...`) showing how MySQL executes a query: access type (ALL = full scan is bad, ref/range = index used), rows examined, and indexes chosen.

269. How can indexes improve performance?

They turn full table scans into direct lookups, speed up joins on indexed keys, and can pre-satisfy ORDER BY. A covering index can answer a query without touching the table at all.

270. What causes a query to run slowly?

Missing indexes, full table scans, functions on indexed columns, joining huge tables before filtering, SELECT *, correlated subqueries, stale statistics, locking, and returning too much data.

271. How can you optimize a JOIN?

Index the join columns, join on the same data types, filter rows before joining (subquery/CTE), start from the smaller table, and avoid joining on expressions.

272. How can you optimize a GROUP BY query?

Filter rows first with WHERE (not HAVING where possible), index the grouping columns, pre-aggregate the smallest possible dataset, and avoid grouping on expressions.

273. Why can functions on indexed columns hurt performance?

`WHERE YEAR(order_date) = 2026` forces the engine to compute the function for every row, so it can't use the index. Rewrite as a range: `WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'`.

274. What is a full table scan?

Reading every row of a table to answer a query (EXPLAIN type = ALL). Fine for tiny tables; a red flag on large ones — usually means a missing/unusable index.

275. What is query optimization?

The process of rewriting queries and adjusting schema/indexes so the database executes them with minimal time and resources — including what the optimizer does automatically and what you do manually.

276. How can you reduce unnecessary data processing?

Select only required columns, apply WHERE filters as early as possible, aggregate before joining, use LIMIT for previews, and avoid DISTINCT unless truly needed.

277. When should you use a CTE versus a subquery?

Use a CTE for readability, multi-step logic, reuse of an intermediate result, or recursion. Use a simple subquery for short one-off filters. Performance is usually equivalent in MySQL 8+.

278. How can you identify performance bottlenecks?

Run EXPLAIN/EXPLAIN ANALYZE on slow queries, check the slow query log, look at rows examined vs. rows returned, monitor locks and buffer usage (SHOW ENGINE INNODB STATUS), and profile with sys/performance_schema.

19. Transactions

★ 279. What is a transaction?

A sequence of SQL operations executed as a single logical unit of work — either all succeed (COMMIT) or all are undone (ROLLBACK). Classic example: transferring money between accounts.

★ 280. What are ACID properties?

Atomicity: all-or-nothing execution. **Consistency**: transactions move the DB from one valid state to another (constraints hold). **Isolation**: concurrent transactions don't interfere. **Durability**: committed changes survive crashes.

281. What is COMMIT?

The command that permanently saves all changes made in the current transaction.

282. What is ROLLBACK?

The command that undoes all uncommitted changes in the current transaction, restoring the previous state.

283. What is SAVEPOINT?

A marker inside a transaction you can partially roll back to: `SAVEPOINT sp1; ... ROLLBACK TO sp1;` — undoing later steps while keeping earlier ones.

284. What is transaction isolation?

The degree to which concurrent transactions are shielded from each other's uncommitted changes. Levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ (MySQL default), SERIALIZABLE.

285. What is atomicity?

A transaction's operations are indivisible: either every statement applies or none do — no half-completed transfers.

286. What is consistency?

Every transaction leaves the database satisfying all rules and constraints (keys, checks, triggers); invalid transactions are rejected/rolled back.

287. What is isolation?

Concurrent transactions behave as if run one at a time — one transaction can't see another's uncommitted intermediate state (depending on the isolation level).

288. What is durability?

Once COMMIT succeeds, the changes are permanent and survive power failures or crashes, typically guaranteed by write-ahead logging.

20. Frequently Asked SQL Coding Questions

★ 289. Find the second-highest salary.

```
-- Option 1
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
-- Option 2
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

★ 290. Find the third-highest salary.

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

★ 291. Find the Nth-highest salary.

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET N-1;
-- or: SELECT salary FROM (SELECT salary, DENSE_RANK() OVER
--   (ORDER BY salary DESC) rnk FROM employees) t WHERE rnk = N;
```

★ 292. Find the highest salary in each department.

```
SELECT department, MAX(salary) AS max_salary
FROM employees GROUP BY department;
```

★ 293. Find employees earning more than the average salary.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

★ 294. Find duplicate records.

```
SELECT email, COUNT(*) AS cnt
FROM employees GROUP BY email HAVING cnt > 1;
```

★ 295. Find employees who don't have a department.

```
SELECT e.* FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;
```

★ 296. Find customers who never placed an order.

```
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

★ 297. Find the top 5 highest-paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

★ 298. Find the number of employees in each department.

```
SELECT department, COUNT(*) AS emp_count
FROM employees GROUP BY department;
```

★ 299. Find departments with more than 5 employees.

```
SELECT department FROM employees
GROUP BY department HAVING COUNT(*) > 5;
```

★ 300. Find the average salary by department.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees GROUP BY department;
```

21. Intermediate SQL Problems

★ 301. Find the second-highest salary in each department.

```
SELECT department, salary FROM (
  SELECT department, salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) rnk
  FROM employees
) t WHERE rnk = 2;
```

★ 302. Find the top 3 salaries in each department.

```
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER
    (PARTITION BY department ORDER BY salary DESC) rn
  FROM employees e
) t WHERE rn <= 3;
```

303. Find employees who earn more than their manager.

```
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

304. Find employees who earn the same salary.

```
SELECT salary, COUNT(*) AS cnt
FROM employees
GROUP BY salary HAVING cnt > 1;
```

305. Find the department with the highest average salary.

```
SELECT department, AVG(salary) AS avg_sal
FROM employees GROUP BY department
ORDER BY avg_sal DESC LIMIT 1;
```

306. Find the department with the highest number of employees.

```
SELECT department, COUNT(*) AS cnt
FROM employees GROUP BY department
ORDER BY cnt DESC LIMIT 1;
```

307. Find customers with more than 3 orders.

```
SELECT customer_id FROM orders
GROUP BY customer_id HAVING COUNT(*) > 3;
```

308. Find customers whose total spending is above average.

```
WITH totals AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders GROUP BY customer_id
)
SELECT * FROM totals
WHERE total > (SELECT AVG(total) FROM totals);
```

309. Find the most expensive product in each category.

```
SELECT * FROM (
  SELECT p.*, ROW_NUMBER() OVER
    (PARTITION BY category ORDER BY price DESC) rn
  FROM products p
) t WHERE rn = 1;
```

310. Find the second-most expensive product in each category.

Same pattern as Q309 with `rn = 2` (use DENSE_RANK if you want ties handled by distinct price).

311. Find the highest-selling product.

```
SELECT product_id, SUM(quantity) AS units_sold
FROM order_items GROUP BY product_id
ORDER BY units_sold DESC LIMIT 1;
```

312. Find the top 3 products by sales.

```
SELECT product_id, SUM(amount) AS revenue
FROM order_items GROUP BY product_id
ORDER BY revenue DESC LIMIT 3;
```

313. Find products that have never been sold.

```
SELECT p.* FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
WHERE oi.product_id IS NULL;
```

314. Find customers who ordered in January but not February.

```
SELECT DISTINCT customer_id FROM orders
WHERE MONTH(order_date) = 1 AND YEAR(order_date) = 2026
  AND customer_id NOT IN (
    SELECT customer_id FROM orders
    WHERE MONTH(order_date) = 2 AND YEAR(order_date) = 2026
  );
```

315. Find customers who ordered every month.

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_FORMAT(order_date, '%Y-%m'))
     = (SELECT COUNT(DISTINCT DATE_FORMAT(order_date, '%Y-%m')) FROM orders);
```

22. Advanced Analytical SQL Questions

★ 316. Calculate month-over-month sales growth.

```
WITH m AS (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, SUM(amount) AS sales
  FROM orders GROUP BY month
)
SELECT month, sales,
  100.0 * (sales - LAG(sales) OVER (ORDER BY month))
        / LAG(sales) OVER (ORDER BY month) AS mom_growth_pct
FROM m;
```

★ 317. Calculate year-over-year sales growth.

```
WITH y AS (
  SELECT YEAR(order_date) AS yr, SUM(amount) AS sales
  FROM orders GROUP BY yr
)
SELECT yr, sales,
  100.0 * (sales - LAG(sales) OVER (ORDER BY yr))
        / LAG(sales) OVER (ORDER BY yr) AS yoy_growth_pct
FROM y;
```

★ 318. Calculate a running total.

```
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM sales;
```

★ 319. Calculate a 7-day moving average.

```
SELECT order_date, amount,
  AVG(amount) OVER (ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma_7d
FROM daily_sales;
```

★ 320. Find the top 3 products in every category.

```
SELECT * FROM (
  SELECT category, product_name, sales,
    ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC) rn
  FROM product_sales
) t WHERE rn <= 3;
```

★ 321. Find the first order of every customer.

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER
    (PARTITION BY customer_id ORDER BY order_date) rn
  FROM orders o
) t WHERE rn = 1;
```

★ 322. Find the latest order of every customer.

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER
    (PARTITION BY customer_id ORDER BY order_date DESC) rn
  FROM orders o
) t WHERE rn = 1;
```

★ 323. Find customers' previous order date.

```
SELECT customer_id, order_date,
  LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
    AS prev_order_date
FROM orders;
```

★ 324. Calculate the number of days between customer orders.

```
SELECT customer_id, order_date,
  DATEDIFF(order_date,
    LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
  ) AS days_since_last_order
FROM orders;
```

★ 325. Find customers who made purchases in consecutive months.

```
WITH m AS (
  SELECT DISTINCT customer_id, DATE_FORMAT(order_date,'%Y-%m') AS ym
  FROM orders
)
SELECT customer_id FROM (
  SELECT customer_id, ym,
    LAG(ym) OVER (PARTITION BY customer_id ORDER BY ym) AS prev_ym
  FROM m
) t
WHERE PERIOD_DIFF(REPLACE(ym,'-',''), REPLACE(prev_ym,'-','')) = 1;
```

326. Find the longest consecutive purchasing streak.

Classic gaps-and-islands: convert purchase months to a period number, subtract ROW_NUMBER to form an island group, then take the max streak length per customer:

```
WITH m AS (
  SELECT DISTINCT customer_id,
         PERIOD_DIFF(DATE_FORMAT(order_date,'%Y%m'),'200001') AS p
  FROM orders
), g AS (
  SELECT customer_id, p,
    p - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY p) AS grp
  FROM m
)
SELECT customer_id, MAX(streak) AS longest_streak FROM (
  SELECT customer_id, grp, COUNT(*) AS streak FROM g
  GROUP BY customer_id, grp
) s GROUP BY customer_id;
```

327. Find customers whose spending increased every month.

```
WITH m AS (
  SELECT customer_id, DATE_FORMAT(order_date,'%Y-%m') AS ym,
         SUM(amount) AS spend
  FROM orders GROUP BY customer_id, ym
), f AS (
  SELECT customer_id,
    SUM(spend <= LAG(spend) OVER w) AS dips
  FROM m WINDOW w AS (PARTITION BY customer_id ORDER BY ym)
)   -- note: first month has NULL lag; handle per data
SELECT customer_id FROM f GROUP BY customer_id
HAVING SUM(dips) = 0;
```

328. Find the percentage contribution of each product to total sales.

```
SELECT product_id, SUM(amount) AS sales,
  100.0 * SUM(amount) / SUM(SUM(amount)) OVER () AS pct_of_total
FROM order_items
GROUP BY product_id;
```

329. Find each department's percentage contribution to company salary.

```
SELECT department, SUM(salary) AS dept_salary,
  100.0 * SUM(salary) / SUM(SUM(salary)) OVER () AS pct
FROM employees GROUP BY department;
```

330. Find the cumulative percentage of sales.

```
WITH m AS (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, SUM(amount) AS sales
  FROM orders GROUP BY month
)
SELECT month, sales,
  100.0 * SUM(sales) OVER (ORDER BY month) / SUM(sales) OVER () AS cum_pct
FROM m;
```

331. Find the median salary.

```
SELECT AVG(salary) AS median_salary FROM (
  SELECT salary,
    ROW_NUMBER() OVER (ORDER BY salary) AS rn,
    COUNT(*) OVER () AS n
  FROM employees
) t
WHERE rn IN (FLOOR((n+1)/2), CEIL((n+1)/2));
```

332. Find the mode of a column.

```
SELECT salary, COUNT(*) AS freq
FROM employees GROUP BY salary
ORDER BY freq DESC LIMIT 1;
```

333. Find the most frequently purchased product.

```
SELECT product_id, COUNT(*) AS times_bought
FROM order_items GROUP BY product_id
ORDER BY times_bought DESC LIMIT 1;
```

334. Find the customer with the highest lifetime value.

```
SELECT customer_id, SUM(amount) AS ltv
FROM orders GROUP BY customer_id
ORDER BY ltv DESC LIMIT 1;
```

335. Find customers whose latest order value is greater than their average order value.

```
WITH ranked AS (
  SELECT customer_id, amount,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) rn,
    AVG(amount) OVER (PARTITION BY customer_id) AS avg_amt
  FROM orders
)
SELECT customer_id FROM ranked
WHERE rn = 1 AND amount > avg_amt;
```

23. Business / Data Analyst SQL Questions

336. How would you calculate monthly revenue?

```
SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,
       SUM(amount) AS revenue
FROM orders
WHERE status = 'Completed'
GROUP BY month ORDER BY month;
```

337. How would you calculate customer retention?

Define a cohort (e.g., customers acquired in month X), then measure the share still active in later months: retention = (active customers from cohort in month N) / (cohort size). Implemented with a cohort CTE (MIN order month) + COUNT(DISTINCT) per activity month (see Q195).

338. How would you calculate customer churn?

Churn rate = customers lost in period / customers at start of period. "Lost" typically means no order in the last X days:

```
SELECT COUNT(*) AS churned FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id
    AND o.order_date >= CURDATE() - INTERVAL 90 DAY
);
```

339. How would you calculate Customer Lifetime Value?

Simple LTV = total revenue per customer: `SUM(amount) GROUP BY customer_id`. Predictive LTV = avg order value × purchase frequency × expected lifespan. Clarify which definition the business uses.

340. How would you identify your top customers?

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders GROUP BY customer_id
ORDER BY total_spent DESC LIMIT 10;
```

341. How would you identify inactive customers?

```
SELECT c.id, MAX(o.order_date) AS last_order
FROM customers c LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id
HAVING last_order IS NULL
    OR last_order < CURDATE() - INTERVAL 90 DAY;
```

342. How would you identify repeat customers?

```
SELECT customer_id FROM orders
GROUP BY customer_id HAVING COUNT(DISTINCT order_id) > 1;
```

343. How would you calculate average order value?

```
SELECT SUM(amount)/COUNT(DISTINCT order_id) AS aov FROM orders;
```

344. How would you calculate conversion rate?

```
SELECT 100.0 *
  COUNT(DISTINCT CASE WHEN purchased = 1 THEN visitor_id END)
  / COUNT(DISTINCT visitor_id) AS conversion_pct
FROM events;
```

345. How would you calculate cancellation rate?

```
SELECT 100.0 *
  SUM(status = 'Cancelled') / COUNT(*) AS cancellation_pct
FROM orders;
```

346. How would you calculate product return rate?

```
SELECT 100.0 *
  SUM(status = 'Returned') / COUNT(*) AS return_pct
FROM order_items;
```

347. How would you identify the best-performing product?

Rank by the agreed KPI (revenue, units, or margin): group order_items by product, order by the KPI descending, take the top row — and state which metric defines "best".

348. How would you identify the worst-performing product?

Same ranking ascending; also consider products with zero sales via LEFT JOIN anti-join, and low margin or high return rate as "worst" definitions.

349. How would you identify the best-performing region?

```
SELECT region, SUM(amount) AS revenue
FROM orders o JOIN customers c ON o.customer_id = c.id
GROUP BY region ORDER BY revenue DESC LIMIT 1;
```

350. How would you identify declining sales?

Compute monthly sales and use LAG() to compare consecutive months; flag months where sales < previous month, or compute a 3-month trend and flag sustained decreases.

351. How would you compare current month sales with previous month?

```
SELECT month, sales,
  LAG(sales) OVER (ORDER BY month) AS prev_sales,
  sales - LAG(sales) OVER (ORDER BY month) AS diff
FROM monthly_sales;
```

352. How would you identify customers whose spending has decreased?

```
WITH m AS (
  SELECT customer_id, DATE_FORMAT(order_date,'%Y-%m') AS ym,
         SUM(amount) AS spend
  FROM orders GROUP BY customer_id, ym
)
SELECT customer_id, ym, spend,
  spend - LAG(spend) OVER (PARTITION BY customer_id ORDER BY ym) AS change
FROM m;  -- negative change = decreased
```

353. How would you calculate the percentage of total sales by category?

```
SELECT category, SUM(amount) AS sales,
  100.0 * SUM(amount) / SUM(SUM(amount)) OVER () AS pct
FROM order_items oi JOIN products p ON p.id = oi.product_id
GROUP BY category;
```

354. How would you find the most profitable category?

```
SELECT category, SUM(oi.amount - p.cost * oi.quantity) AS profit
FROM order_items oi JOIN products p ON p.id = oi.product_id
GROUP BY category ORDER BY profit DESC LIMIT 1;
```

355. How would you find the most active customers?

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
WHERE order_date >= CURDATE() - INTERVAL 30 DAY
GROUP BY customer_id ORDER BY order_count DESC LIMIT 10;
```

24. SQL + Real-World Scenarios

356. Customers and orders tables: find customers who haven't purchased anything.

```
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

357. Employee table: find employees who earn more than their manager.

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

358. Sales data: find the top 3 products every month.

```
SELECT * FROM (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, product_id,
         SUM(amount) AS sales,
         ROW_NUMBER() OVER (
           PARTITION BY DATE_FORMAT(order_date,'%Y-%m')
           ORDER BY SUM(amount) DESC) rn
  FROM orders GROUP BY month, product_id
) t WHERE rn <= 3;
```

359. Website traffic data: calculate daily active users.

```
SELECT DATE(event_time) AS day, COUNT(DISTINCT user_id) AS dau
FROM events GROUP BY day ORDER BY day;
```

360. Find users who logged in for 3 consecutive days.

```
WITH d AS (
  SELECT DISTINCT user_id, DATE(login_time) AS dt FROM logins
), g AS (
  SELECT user_id, dt,
    DATE_SUB(dt, INTERVAL ROW_NUMBER() OVER
      (PARTITION BY user_id ORDER BY dt) DAY) AS grp
  FROM d
)
SELECT user_id FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

361. How would you calculate monthly active users?

```
SELECT DATE_FORMAT(event_time,'%Y-%m') AS month,
       COUNT(DISTINCT user_id) AS mau
FROM events GROUP BY month ORDER BY month;
```

362. How would you identify users who became inactive?

```
SELECT user_id, MAX(event_time) AS last_seen
FROM events GROUP BY user_id
HAVING last_seen < NOW() - INTERVAL 30 DAY;
```

363. How would you calculate customer retention after 30 days?

For each new customer (first activity date = signup day), check whether they have any activity in the 30 days after: retention = customers active again within 30 days / new customers in the cohort.

```
WITH firsts AS (
  SELECT customer_id, MIN(DATE(order_date)) AS first_dt
  FROM orders GROUP BY customer_id
)
SELECT first_dt AS cohort,
  100.0 * SUM(EXISTS (
    SELECT 1 FROM orders o2
    WHERE o2.customer_id = f.customer_id
      AND DATE(o2.order_date) BETWEEN f.first_dt + INTERVAL 1 DAY
                                  AND f.first_dt + INTERVAL 30 DAY
  )) / COUNT(*) AS retention_30d_pct
FROM firsts f GROUP BY cohort;
```

364. How would you calculate average order value?

```
SELECT SUM(amount)/COUNT(DISTINCT order_id) AS aov FROM orders;
```

365. How would you calculate revenue growth?

Compare revenue between periods using LAG(): (current − previous) / previous × 100. Works for monthly (MoM), quarterly, or yearly (YoY) grain — see Q316/Q317.

366. How would you find the first purchase made by each customer?

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER
    (PARTITION BY customer_id ORDER BY order_date) rn
  FROM orders o
) t WHERE rn = 1;
```

367. How would you identify repeat purchases?

```
SELECT customer_id, product_id, COUNT(*) AS times_bought
FROM order_items oi JOIN orders o ON o.id = oi.order_id
GROUP BY customer_id, product_id
HAVING times_bought > 1;
```

368. How would you identify duplicate transactions?

```
SELECT customer_id, amount, transaction_time, COUNT(*) AS cnt
FROM transactions
GROUP BY customer_id, amount, transaction_time
HAVING cnt > 1;
```

369. How would you identify transactions that occurred more than once?

Same GROUP BY + HAVING COUNT(*) > 1 pattern on the business key that should be unique (e.g., transaction reference number).

370. How would you find the most popular product by month?

```
SELECT month, product_id, units FROM (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, product_id,
         SUM(quantity) AS units,
         ROW_NUMBER() OVER (
           PARTITION BY DATE_FORMAT(order_date,'%Y-%m')
           ORDER BY SUM(quantity) DESC) rn
  FROM orders GROUP BY month, product_id
) t WHERE rn = 1;
```

371. Find the highest revenue-generating customer in each region.

```
SELECT region, customer_id, revenue FROM (
  SELECT c.region, o.customer_id, SUM(o.amount) AS revenue,
    ROW_NUMBER() OVER (PARTITION BY c.region ORDER BY SUM(o.amount) DESC) rn
  FROM orders o JOIN customers c ON c.id = o.customer_id
  GROUP BY c.region, o.customer_id
) t WHERE rn = 1;
```

25. MySQL-Specific Questions

★ 372. What is MySQL?

A free, open-source RDBMS (now owned by Oracle) using SQL. Very popular for web apps and analytics; known for speed, reliability, and ease of use. Runs client-server, default port 3306.

373. What are MySQL data types?

Numeric (INT, BIGINT, DECIMAL, FLOAT, DOUBLE), String (CHAR, VARCHAR, TEXT, ENUM), Date/Time (DATE, DATETIME, TIMESTAMP, TIME, YEAR), and JSON.

374. Difference between CHAR and VARCHAR?

CHAR is fixed-length (padded with spaces, fast for constant-size data like country codes); VARCHAR is variable-length and stores only actual characters plus length. VARCHAR saves space; CHAR is slightly faster for fixed data.

375. Difference between DATE, DATETIME, and TIMESTAMP?

DATE stores 'YYYY-MM-DD' only. DATETIME stores date+time as-is ('YYYY-MM-DD HH:MM:SS'), range 1000–9999. TIMESTAMP also stores date+time but in UTC with timezone conversion on read/write, range 1970–2038, and can auto-update.

376. What is AUTO_INCREMENT?

A column attribute that automatically generates sequential numbers on INSERT — the standard way to create surrogate primary keys in MySQL.

377. What is LIMIT?

Restricts the number of rows returned: `LIMIT 10`, or `LIMIT 20, 10` (offset 20, take 10) for pagination.

378. What is IFNULL()?

`IFNULL(expr, alt)` returns expr unless it's NULL, then returns alt. Two arguments only, MySQL-specific.

379. What is COALESCE()?

Returns the first non-NULL argument from a list of any length; ANSI-standard and portable across databases.

380. What is STR_TO_DATE()?

Parses a string into a date using a format: `STR_TO_DATE('08-10-2026','%d-%m-%Y')` — essential when importing text dates.

381. What is DATE_FORMAT()?

Formats a date as a string: `DATE_FORMAT(order_date,'%Y-%m')` → '2026-10'. Common codes: %Y year, %m month, %d day, %H %i %s time.

382. What is TIMESTAMPDIFF()?

Difference between two datetime values in a chosen unit: `TIMESTAMPDIFF(DAY, start, end)`. Units: SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, YEAR.

383. What is GROUP_CONCAT()?

Aggregates values from grouped rows into one delimited string: `SELECT dept, GROUP_CONCAT(name) FROM employees GROUP BY dept;`

384. What is SUBSTRING_INDEX()?

Returns the part of a string before/after the Nth occurrence of a delimiter: `SUBSTRING_INDEX(email,'@',-1)` → domain. Great for splitting columns.

385. What is REGEXP?

Regular-expression matching: `WHERE email REGEXP '^[a-z0-9._%+-]+@'` — more powerful than LIKE for pattern validation.

386. What is EXPLAIN?

Shows MySQL's execution plan for a query: access type, indexes considered/used, rows examined — the first tool for diagnosing slow queries.

387. What is LOAD DATA INFILE?

A fast bulk-import command that loads a text/CSV file into a table directly: `LOAD DATA INFILE 'data.csv' INTO TABLE t FIELDS TERMINATED BY ',';` — far faster than row-by-row INSERTs.

388. What is ONLY_FULL_GROUP_BY?

A default SQL mode in MySQL 5.7+/8 requiring every non-aggregated SELECT column to appear in GROUP BY, preventing ambiguous results.

389. Difference between VARCHAR and TEXT?

VARCHAR has a defined max length (up to 65,535 bytes row limit), can have defaults and be fully indexed. TEXT stores up to 65,535 chars (MEDIUMTEXT/LONGTEXT larger), can't have a default value, and needs a prefix length to be indexed.

390. What are stored procedures?

Precompiled SQL routines stored in the database, executed with CALL. They encapsulate reusable logic, accept parameters, and reduce network round-trips.

391. What are triggers?

SQL code that automatically executes BEFORE or AFTER INSERT/UPDATE/DELETE on a table — used for auditing, enforcing complex rules, or maintaining derived data.

392. What are functions in MySQL?

Reusable routines that return a single value and can be called inside SQL expressions (`SELECT my_func(col)`). Distinct from stored procedures: functions return values; procedures perform actions and can return result sets.

26. Skills Checklist & The 25 Questions to Prepare First

Must Know (fresher/intermediate Data Analyst level)

SELECT • WHERE • DISTINCT • ORDER BY • LIMIT • GROUP BY • HAVING • COUNT, SUM, AVG, MIN, MAX • CASE • NULL handling • INNER / LEFT / RIGHT JOIN • Subqueries • CTEs • UNION / UNION ALL • ROW_NUMBER, RANK, DENSE_RANK • LAG / LEAD • Date functions • String functions • Duplicate detection • Data cleaning • Second-highest salary • Top N records • Top N per group • Running total • Month-over-month analysis

Good to Know

Recursive CTEs • Views • Indexes • Query optimization • Execution plans • Transactions • ACID • Stored procedures • Triggers • Advanced window functions

The 25 Questions to Prepare First

1. What is SQL?
2. SQL vs MySQL?
3. Primary Key vs Foreign Key?
4. WHERE vs HAVING?
5. DELETE vs DROP vs TRUNCATE? — DELETE removes rows (DML, can roll back, optional WHERE); TRUNCATE removes all rows fast (DDL, resets AUTO_INCREMENT, no WHERE); DROP deletes the whole table structure.
6. What is GROUP BY?
7. INNER JOIN vs LEFT JOIN?
8. What are aggregate functions?
9. COUNT(*) vs COUNT(column)?
10. What is NULL and how do you handle it?
11. What is a subquery?
12. What is a CTE?
13. CTE vs subquery?
14. What is a CASE statement?
15. What is a window function?
16. RANK vs DENSE_RANK vs ROW_NUMBER?
17. What is PARTITION BY?
18. Find the second-highest salary (Q289).
19. Find the top 3 employees in each department (Q302).
20. Find duplicate records (Q294).
21. Find customers who never placed an order (Q296).
22. Find employees earning more than the average salary (Q293).
23. Calculate a running total (Q318).
24. Calculate month-over-month sales (Q316).
25. Find the highest-selling product/category (Q311).

> *Preparation tip: interviews move from syntax recall to analytical problem-solving. Put extra emphasis on JOINs, GROUP BY/HAVING, CTEs, CASE, subqueries, and window functions — and practice writing each coding answer by hand.*